Práctica 3

1)DyV por SUSTRACCIÓN

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Las complejidades correspondientes a los tres algoritmos de Sustracción iniciales son:

- Sustracción 1: O(n)

A=1, b=1 y k=0. Por ende, al estar trabajando con DyV con sustracción y a=1, seguimos la fórmula Complejidad = O(n^k+1) y sacamos que su complejidad es de O(n).

Respecto a si los tiempos obtenidos van acorde a la complejidad, basándome en los que obtenemos antes de que salte una excepción, podemos apreciar que estos van multiplicándose, aproximadamente, por 2. Por lo tanto, sí son acordes a la complejidad.

- Sustracción 2: O(n^2)

A=1, b=1 y k=1. Por ende, al estar trabajando con DyV con sustracción y a=1, seguimos la fórmula Complejidad = O(n^k+1) y sacamos que su complejidad es de O(n^2).

Respecto a si los tiempos obtenidos van acorde a la complejidad o no, basándonos, como en el caso anterior, hasta antes de que salte una excepción, se pueden apreciar como la multiplicación de tiempos rondan aproximadamente a ir multiplicándose por 4, por ende, es acorde a la complejidad.

- Sustracción 3: O(2^n)

A=2, b=1 y k=0. Por ende, al estar trabajando con DyV con sustracción y a>1, seguimos la fórmula Complejidad = O(a^n/b), sacamos que su complejidad es de O(2^n).

Respecto a si los tiempos obtenidos van acorde a la complejidad, podemos ver que estos se van multiplicando por 2 y que, a diferencia que con Sustracción1, no salta una excepción, pues no se llega a desbordar la pila.

Contestar: ¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustraccion1 y Sustraccion2? ¿por qué sucede eso?

En el caso de Sustracción1, con un número de repeticiones nVeces = 1, el último tamaño de n tomado previo a la excepción es n = 8000.

Por otro lado, Sustracción2, con el mismo número de nVeces que el anterior, también salta la excepción con el último tiempo tomado siendo el correspondiente a n = 8000.

La razón por la que sucede esto es porque con estas complejidades llega un punto en el que la pila se ve desbordada. Y en Sustracción3 no sucede esto gracias a que, al ser el incremento por O(2^n) uno tan grande, no llega a desbordarse.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustraccion3 para n=80? (realizar regla de tres)

Sabiendo que la complejidad de este algoritmo es de O(2^n), y que con nVeces=1 el tiempo tomado para n=30 es de 75662 milisegundos

N = 80 -> 201,76 segundos

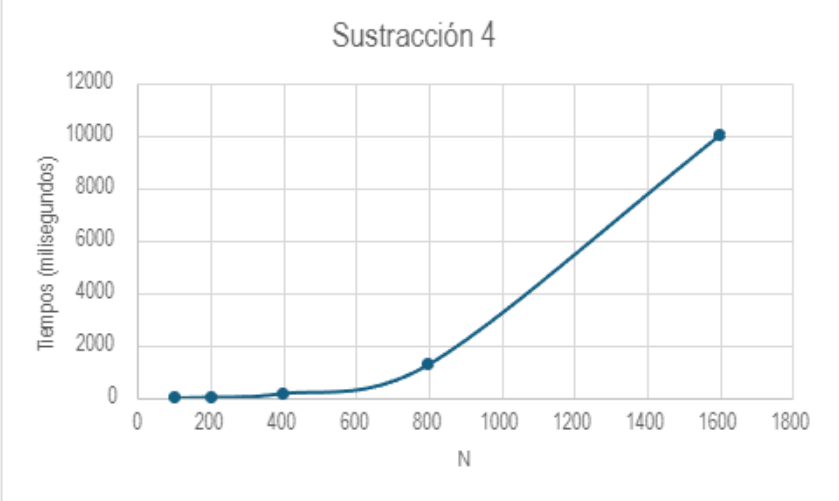
Implementar una clase Sustraccion4.java con una complejidad O(n^3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en msg.) para n=100, 200, 400, 800, … (hasta FdT).

- Sustracción 4: O(n^3)

Para obtener un algoritmo DyV por sustracción de tal complejidad, yo opté por los valores A=1, b=2 y k=2. Por ende, al estar trabajando con DyV con sustracción y A=1, seguimos la fórmula Complejidad = O(n^k+1) y sacamos que su complejidad es de O(n^3).

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	Tiempos (milisegundos)
100	2,64
200	20,4
400	161
800	1272
1600	10032
3200	FdT



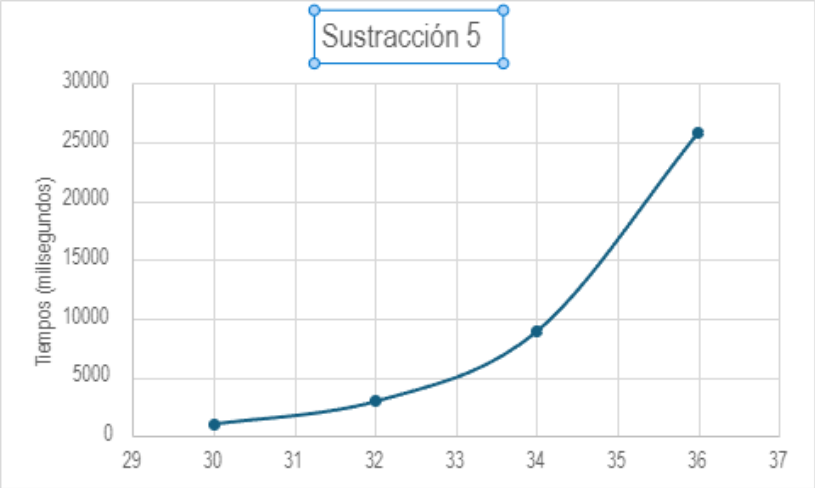
Implementar una clase Sustraccion5.java con una complejidad O(3^n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en msg.) para n=30, 32, 34, 36, … (hasta FdT).

- Sustracción 5: O(3^n/2)

Para obtener un algoritmo DyV por sustracción de tal complejidad, yo opté por los valores A=3, b=2 y k=1. Por ende, al estar trabajando con DyV con sustracción y A>1, seguimos la fórmula Complejidad = O(a^n/b), sacamos que su complejidad es de O(3^n/2).

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	Tiempos (milisegundos)
30	1005
32	2930
34	8891
36	25827
38	FdT



2)DyV por DIVISIÓN

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Las complejidades correspondientes a las tres clases de División iniciales son:

- División 1: O(n)

A=1, b=3 y k=1. Por ende, al estar trabajando con DyV con división y a<b^k, seguimos la fórmula Complejidad = O(n^k) y sacamos que su complejidad es de O(n).

Respecto a si los tiempos obtenidos van acorde a la complejidad, se ve que los tiempos van multiplicándose, aproximadamente, por 2. Por lo tanto, sí son acordes a la complejidad.

- División 2: O(n*logn)

A=2, b=2 y k=1. Por ende, al estar trabajando con DyV con división y a=b^k, seguimos la fórmula Complejidad = O(n^k*logn) y sacamos que su complejidad es de O(n*logn).

Respecto a si los tiempos obtenidos van acorde a la complejidad, estos se van multiplicando por 2, más o menos aproximado según la iteración, lo cual era de esperar sabiendo que tratábamos con un O(n*logn).

- División 3: O(n)

A=2, b=2 y k=0. Por ende, al estar trabajando con DyV con división y a>b^k, seguimos la fórmula Complejidad = O(n^(log_b_a)) y sacamos que su complejidad es de O(n^1) y, por ende, O(n).

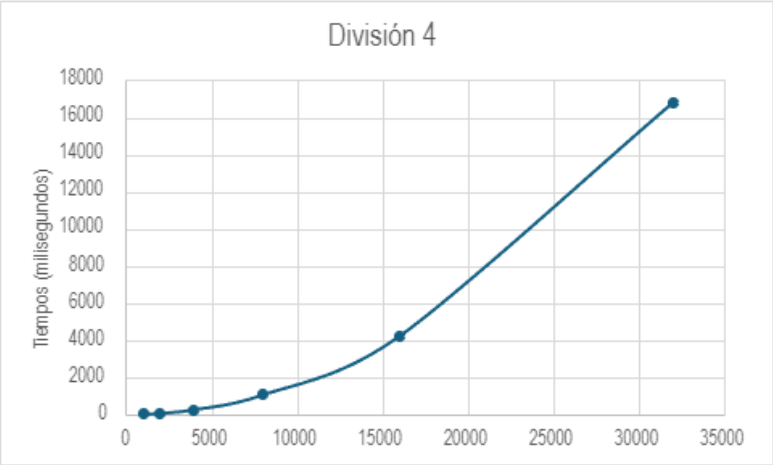
Respecto a si los tiempos obtenidos van acorde a la complejidad, como en el caso del primero, volvemos a lidiar con un O(n), y al obtener tiempos que se van multiplicando por 2, sabemos que se cumple lo esperado de esta complejidad.

Implementar una clase Division4.java con una complejidad O(n^2) (con a<b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en msg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

Para obtener un algoritmo DyV por división de tal complejidad (con a<b^k), yo opté por los valores A=1, b=3 y k=2. Por ende, al estar trabajando con DyV con división y a<b^k, seguimos la fórmula Complejidad = O(n^k) y sacamos que su complejidad es de O(n^2).

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	Tiempos (milisegundos)
1000	16,9
2000	68
4000	268
8000	1060
16000	4219
32000	16799
64000	FdT

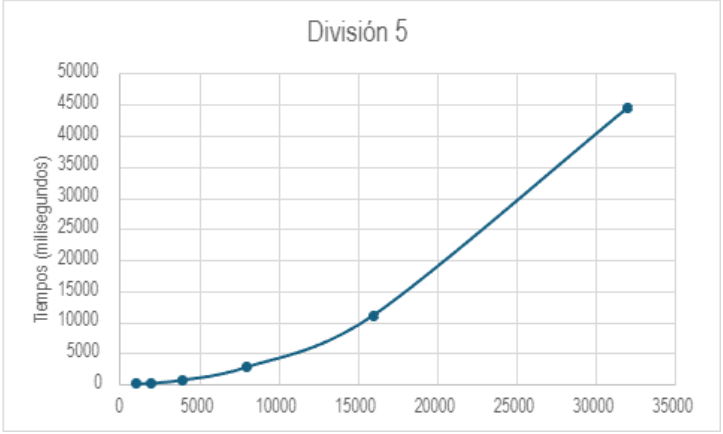


Implementar una clase Division5.java con una complejidad O(n^2) (con a>b^k) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en msg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

Para obtener un algoritmo DyV por división de tal complejidad (con a>b^k), yo opté por los valores A=4, b=2 y k=1. Por ende, al estar trabajando con DyV con división y a>b^k, seguimos la fórmula Complejidad = O(n^(log_b_a)) y sacamos que su complejidad es, al ser el resultado del logaritmo planteado como exponente 2, de O(n^2).

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	Tiempos (milisegundos)
1000	45,1
2000	177
4000	694
8000	2761
16000	11106
32000	44456
64000	FdT



3)DOS EJEMPLOS BÁSICOS

Tras analizar la complejidad de los diversos algoritmos que hay dentro de las dos clases (VectorSuma y Fibonacci), ejecutarlas y tras poner en una tabla los tiempos obtenidos, comparar la eficiencia de cada algoritmo.

- VectorSuma

Esto medido con un ordenador de procesador Intel i5-12400 2.5GHz:

n = 100	Tiempos (nanosegundos)
Método 1 (iterativo O(n))	433
Método 2 (recursivo Sust O(n))	1796
Método 3 (recursivo Div O(n))	3462

- Fibonacci.java

Esto medido con un ordenador de procesador Intel i5-12400 2.5GHz:

n=40	Tiempos (nanosegundos)
Método 1 (iterativo O(n))	220
Método 2 (iterativo O(n))	323
Método 3 (recursivo Sust O(n))	606
Método 4 (recur Sust O(1.6^n))	4256

4)PRÁCTICA DyV

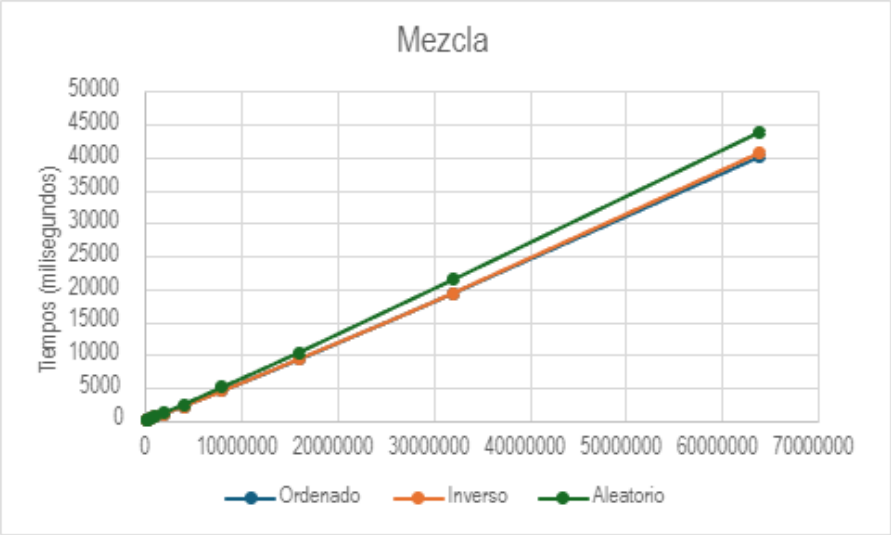
Implementar una clase Mezcla.java que ordene los elementos de un vector.

Implementar una clase MezclaTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla (análoga a las vistas en la práctica 2):

Esto medido con un ordenador de procesador Intel i5-12400 2.5GHz:

N	t ordenado	t inverso	t aleatorio
31250	FdF	FdF	FdF
62500	FdF	FdF	FdF
125000	54	56	62
250000	113	114	129
500000	235	237	266
1000000	493	496	563
2000000	1029	1031	1157

4000000	2154	2152	2409
8000000	4494	4461	5042
16000000	9253	9323	10315
32000000	19264	19319	21366
64000000	40120	40725	43864



Rellene la tabla siguiente, trasladando a ella los tiempos obtenidos para el Rápido en el caso aleatorio en la práctica 2 y los obtenidos aquí para el Mezcla en el caso aleatorio. ¿Qué constante le sale como comparación de ambos algoritmos?

Esto medido con un ordenador de procesador Intel i5-12400 2.5GHz:

n	t1(mezcla)	t2(rápido)	t1/t2
250000	134	125	1,072
500000	275	302	0,910596026
1000000	567	671	0,845007452
2000000	1181	1561	0,756566304
4000000	2484	4191	0,59269864
8000000	5081	12580	0,403895072
16000000	10535	42189	0,24970964
32000000	21699	FdT	-
64000000	44358	FdT	-

Y, tras haber realizado el cambio en la clase Vector.java para corregir el “desliz” que ocurría en el caso Aleatorio de Rápido, obtenemos una tabla mucho más acorde a lo esperado:

Esto medido con un ordenador de procesador Intel i5-12400 2.5GHz:

n	t1(mezcla)	t2(rápido)	t1/t2
250000	134	126	1,063492063
500000	275	255	1,078431373
1000000	567	544	1,042279412
2000000	1181	1130	1,045132743
4000000	2484	2398	1,035863219
8000000	5081	4727	1,074888936
16000000	10535	10584	0,99537037
32000000	21699	22368	0,970091202
64000000	44358	48261	0,919127246

